CODENTRAA

Master Project Guide — Complete Development Process

From Idea → Requirements → Design → Development → Testing → Deployment

Version 1.0

Prepared By: Muhammad Atif

Date: August 2026

1. What Is This Document?

This is the master guide for understanding the complete Codentraa project. It combines the meaning and purpose of the first nine project documents created during the planning and design process. It is intentionally written as a learning and reference document, so a developer can read it from beginning to end and understand what Codentraa is, what will be built, why each document exists, and how the documents connect.

2. Project Identity

Item	Definition
Project Name	Codentraa
Project Type	SaaS web application
Primary Concept	Project and team management platform for organizations/teams
Frontend	Next.js / React
Backend	ASP.NET Core Web API / C#
Database	SQL Server with Entity Framework Core
API Style	REST API with JSON
Design Tool	Figma
API Documentation	OpenAPI / Swagger
Source Control	Git / GitHub

3. Codentraa — What Are We Actually Building?

Codentraa is planned as a SaaS platform where an organization creates a workspace, invites its team, creates projects, creates and assigns tasks, collaborates through comments and attachments, receives notifications, views dashboards/reports, and manages its subscription.

The basic product idea is:

Organization
↓
Invite Team
↓
Create Projects
↓
Create & Assign Tasks
↓
Collaborate
↓
Track Progress
↓
View Reports

↓
Manage Subscription

4. Why Is Codentraa a SaaS?

SaaS means Software as a Service. Instead of installing Codentraa separately for every customer, the application is hosted as an online service. Different organizations can use the same platform while their data remains logically isolated.

Example:

Codentraa Platform
│
├── Organization A → its users/projects/tasks
├── Organization B → its users/projects/tasks
└── Organization C → its users/projects/tasks

Each organization operates in its own tenant/workspace context.

The SaaS model also allows subscription plans and usage limits. Billing is part of the product model, but the core product should be stable before advanced billing features are prioritized.

5. Main Features of Codentraa

Feature	What It Does
Authentication	Register, login, email verification, password recovery and secure access.
Organizations	Create and manage a workspace/tenant.
Members & Roles	Invite users and manage Owner, Admin, Manager, Member and Client access.
Projects	Create, edit, view, archive and manage projects.
Tasks	Create, assign, update status, manage details and track work.
Collaboration	Comments, attachments and activity.
Notifications	Inform users about relevant actions/events.
Dashboard	Show project/task/team summaries.
Reports	Show progress, workload and overdue-task information.
Subscription	Plans, usage limits and subscription management.
Security	Authentication, authorization and tenant isolation.

6. The Complete Development Journey

IDEA
↓
1. SRS
↓
2. User Stories
↓
3. Use Cases
↓
4. Use Case Diagram
↓
5. Database / ERD
↓
6. System Architecture
↓
7. API Design
↓
8. Figma / UI Design
↓
9. Development
↓
10. Testing
↓
11. Deployment
↓
12. Maintenance

The first nine stages have already been documented. The remaining stages are testing, deployment and maintenance.

7. Document 1 — SRS

What is SRS?

SRS means Software Requirements Specification. It answers the first major question: “What should Codentraa do?” It records functional requirements, non-functional requirements, users, system scope, assumptions and constraints.

In simple words

SRS is the project's agreement/blueprint about what the software is supposed to provide.

Codentraa example

Requirement: A Manager should be able to create a project.

Reference document

Codentraa Complete SRS Document — created earlier in this project.

8. Document 2 — User Stories & Acceptance Criteria

User Stories describe requirements from the user's point of view. Acceptance Criteria define what must be true for the story to be considered complete.

Example:

As a Manager,
I want to create a project,
so that I can organize work for my team.

Acceptance Criteria:
• Manager can open Create Project.
• Required fields are validated.
• Authorized manager can create the project.
• Project appears in the project list.
• Unauthorized users cannot create it.

Reference document

User Stories / Acceptance Criteria content used as the input for the Use Case and development documents.

9. Document 3 — Use Cases

A Use Case explains how an actor and the system interact to achieve a specific goal. It goes deeper than a User Story.

User Story:
“Manager wants to create a project.”

Use Case:
1. Manager opens Projects.
2. Manager selects Create Project.
3. System displays the form.
4. Manager enters information.
5. System validates it.
6. System checks permission.
7. System checks plan limits.
8. System saves the project.
9. System returns success.

Reference document

Codentraa_Integrated_Steps_3_4_5_6.docx — Use Case section.

10. Document 4 — Use Case Diagram

The Use Case Diagram is the visual summary of actors and major system functions. It answers: “Who interacts with which parts of Codentraa?”

Actors:
Owner

Admin
Manager
Member / Developer
Client
Payment Provider
Email Service

Major functions:
Authentication
Organizations
Members
Projects
Tasks
Collaboration
Notifications
Reports
Billing

Reference document

Codentraa_Integrated_Steps_3_4_5_6.docx — Use Case Diagram section.

11. Document 5 — Database / ERD

The ERD translates the requirements into data. It answers: “What information does Codentraa need to store, and how is that information related?”

Core data:

User
Organization
OrganizationMember
Role
Project
ProjectMember
Task
Comment
Attachment
Notification
Plan
Subscription
Payment
AuditLog

Main relationship:

Organization
↓
Projects
↓
Tasks

↓
Comments / Attachments

A major SaaS requirement is tenant isolation: organization-owned records must be associated with the correct organization, and the backend must prevent users from accessing another organization's data.

Reference document

Codentraa_Integrated_Steps_3_4_5_6.docx — Database / ERD section.

12. Document 6 — System Architecture

Architecture answers: “How will the technical parts of Codentraa communicate?”

User
↓
Browser
↓
Next.js Frontend
↓ HTTPS / JSON
ASP.NET Core Web API
↓
Business Logic
↓
Entity Framework Core
↓
SQL Server

External services:
Email
File Storage
Payment Provider
Logging / Monitoring

The frontend must not connect directly to SQL Server. Backend services control authorization, validation, business rules and data access.

Reference document

Codentraa_Integrated_Steps_3_4_5_6.docx — System Architecture section.

13. Document 7 — API Design

API Design defines the contract between the frontend and backend. It answers: “How will the application request and receive data?”

Example:

Create Project
POST /api/v1/organizations/{organizationId}/projects

Frontend

↓
API
↓
Authentication
↓
Authorization
↓
Business Rules
↓
SQL Server
↓
JSON Response
↓
Frontend

The API document defines endpoints, methods, request/response formats, status codes, authorization, validation, pagination, security and feature-specific APIs.

Reference document

Codentraa_Step_7_API_Design_and_Specification.docx.

14. Document 8 — Figma / UI Design

Figma converts the functional requirements and API-backed use cases into a visual user interface. It answers: “What will the user see and how will they interact with it?”

Main Figma areas:

Authentication
Dashboard
Projects
Tasks
Team / Members
Reports
Notifications
Billing
Settings

Figma should also contain reusable components, responsive layouts, loading/empty/error/success states, role-aware UI behavior, prototypes and developer handoff information.

Reference document

Codentraa_Step_8_Figma_UI_Design_Specification.docx.

15. Document 9 — Development Process

The Development Process document explains how the designs and requirements become working code. It answers: “How will we actually build Codentraa?”

Recommended development order:

1. Project setup
2. Database foundation
3. Backend foundation
4. Authentication & authorization
5. Organization
6. Members
7. Projects
8. Tasks
9. Collaboration
10. Dashboard
11. Reports
12. SaaS billing
13. Testing/security hardening

Reference document

Codentraa_Step_9_Development_Process_and_Implementation_Plan.docx.

16. How All 9 Documents Connect

SRS
“What are we building?”
↓
USER STORIES
“What does each user need?”
↓
USE CASES
“How does each feature work?”
↓
USE CASE DIAGRAM
“Who interacts with which feature?”
↓
ERD
“What data do we need?”
↓
ARCHITECTURE
“How do technical components communicate?”
↓
API
“How does frontend talk to backend?”
↓
FIGMA
“What does the user interface look like?”
↓
DEVELOPMENT
“How do we implement all of it?”

17. One Complete Example — Create Project

This example shows how one feature travels through the entire process.

Stage	What Happens
SRS	Requirement: authorized users must be able to create projects.
User Story	As a Manager, I want to create a project so that I can organize team work.
Use Case	Manager opens Projects → Create Project → enters data → submits → system validates/authorizes → saves.
Use Case Diagram	Manager is connected to the Create Project use case.
ERD	Project belongs to Organization and stores required project information.
Architecture	Next.js → ASP.NET Core API → Business Logic → EF Core → SQL Server.
API	POST /api/v1/organizations/{organizationId}/projects.
Figma	Create Project screen, form, validation, loading, success and error states.
Development	Implement entity → migration → service → controller → frontend form → API integration → tests.

18. Another Complete Example — Login

User opens Login
↓
Figma defines screen
↓
POST /api/v1/auth/login
↓
API validates credentials
↓
Authentication succeeds
↓
User receives authenticated session/token
↓
Frontend opens Dashboard
↓
Dashboard requests authorized data

This demonstrates why the API, Figma and backend are designed before implementation: the developer already knows what the screen should do and which backend contract it uses.

19. What We Are Building First — MVP

Do not attempt to build every advanced feature at once. The first working Codentraa MVP should focus on the core workflow.

MVP CORE

Register / Login
↓
Create Organization
↓
Invite Members
↓
Create Project
↓
Create / Assign Tasks
↓
Update Task Status
↓
Comments
↓
Dashboard

After this core workflow is stable, expand into notifications, reports, advanced collaboration, subscription management and other SaaS capabilities.

20. What Happens After the 9 Completed Processes?

CURRENT POSITION

Requirements + Design + Development Plan
↓
Actual Coding
↓
Testing
↓
Deployment
↓
Production Codentraa
↓
Monitoring / Maintenance

- Step 10 — Testing: verify functionality, API behavior, security, integration, responsiveness and user acceptance.
- Step 11 — Deployment: configure production infrastructure, database, environment variables, domain, HTTPS and release.
- Step 12 — Maintenance: monitor errors/performance, fix bugs, update dependencies and deliver new features.

21. Practical Rule for Development

Build one feature completely before moving to the next. Do not build the entire frontend first and the entire backend later.

Correct approach:

Feature
↓
Database
↓
Backend/API
↓
Frontend
↓
Integration
↓
Testing
↓
Feature Complete
↓
Next Feature

22. Your Codentraa Development Order

1. Set up Git, Next.js, ASP.NET Core and SQL Server.

2. Implement database entities and migrations.

3. Implement authentication and authorization.

4. Implement organization/tenant management.

5. Implement members and invitations.

6. Implement projects.

7. Implement tasks.

8. Implement comments and attachments.

9. Implement notifications.

10. Implement dashboard.

11. Implement reports.

12. Implement SaaS plans/subscriptions.

13. Complete security and integration testing.

14. Deploy production.

23. Quick Reference — What Each Document Means

Document	Main Question	Output
SRS	What should we build?	Requirements
User Stories	What does the user need?	User-focused requirements
Use Cases	How should a feature behave?	Detailed interaction
Use Case Diagram	Who interacts with what?	Visual actor/function model
ERD	What data do we store?	Database model
Architecture	How does the system work technically?	Technical structure
API Design	How do frontend/backend communicate?	API contract
Figma	What does the user see?	UI/prototype
Development Plan	How do we build it?	Implementation process

24. Document References Created for Codentraa

Reference	Document	Purpose
Reference 01	Complete Codentraa SRS Document	Requirements and project scope.
Reference 02	Codentraa_Integrated_Steps_3_4_5_6.docx	Use Cases, Use Case Diagram, Database/ERD and System Architecture.
Reference 03	Codentraa_Step_7_API_Design_and_Specification.docx	Complete API design and endpoint specification.
Reference 04	Codentraa_Step_8_Figma_UI_Design_Specification.docx	Figma/UI design blueprint.
Reference 05	Codentraa_Step_9_Development_Process_and_Implementation_Plan.docx	Development process and implementation plan.

25. Final Mental Model

Think of Codentraa development like constructing a building:

SRS
= What building do we need?

User Stories
= What do the people using it need?

Use Cases
= How will people use it?

Use Case Diagram
= Who uses which areas?

ERD
= Where will information be stored?

Architecture
= How are the building's systems connected?

API
= How do the systems communicate?

Figma
= What will the building look and feel like?

Development
= Actually construct it.

Testing
= Inspect everything before opening.

Deployment
= Open the building to users.

Maintenance
= Keep it working and improve it.

26. Final Summary

Codentraa is a multi-tenant SaaS project and team management platform. The first nine documents are not separate projects; they are different views of the same project. The SRS defines the requirements, user stories define user needs, use cases define behavior, diagrams define structure, ERD defines data, architecture defines technical communication, API design defines the backend contract, Figma defines the interface, and the development plan explains implementation. Together they form the blueprint for building the real Codentraa application.

The next practical action is to begin Step 9 implementation with the development environment, database foundation and backend setup, then build authentication and the core organization → members → projects → tasks workflow feature-by-feature.

CODENTRAA

Integrated Design Flow — Steps 3, 4, 5 & 6

Use Cases → Use Case Diagram → Database / ERD → System Architecture

Version 1.0

Prepared By: Muhammad Atif

Date: August 2026

1. Purpose

This document combines Steps 3, 4, 5, and 6 of the Codentraa development process into one continuous design phase. It converts approved User Stories and Acceptance Criteria into detailed system behavior, a visual actor/system model, a database structure, and a technical architecture.

2. Position in the Overall Process

SRS → User Stories & Acceptance Criteria → USE CASES → USE CASE DIAGRAM → DATABASE / ERD → SYSTEM ARCHITECTURE → API Design → Figma/UI → Development → Testing → Deployment

3. Integrated Feature Flow

User Story → Use Case → Actor/Interaction → Data Required → Database Entity → API/Service → UI → System Architecture

Example: “Manager creates a project” becomes a Create Project Use Case, appears in the Use Case Diagram, requires Project/Organization/User data in the ERD, and is implemented through the frontend, API, business layer, and database within the system architecture.

4. Step 3 — Detailed Use Cases

A Use Case explains exactly how an actor performs an operation in Codentraa and how the system responds. Use Cases are derived from the User Stories and Acceptance Criteria.

4.1 Standard Use Case Template

- Use Case ID
- Use Case Name
- Primary Actor
- Secondary Actors / External Systems
- Goal
- Preconditions
- Trigger
- Main Success Flow
- Alternative Flows
- Exception / Error Flows
- Postconditions
- Business Rules
- Related User Story
- Acceptance Criteria

4.2 Example — UC-PROJ-001 Create Project

Primary Actor: Manager

Goal: Create a project inside the active organization.

Preconditions: User is authenticated; user belongs to the organization; user has project creation permission; organization has not exceeded the applicable plan limit.

Trigger: Manager selects “Create Project”.

Main Success Flow:

1. Manager opens the Projects page.
2. System verifies authentication and organization context.
3. Manager selects Create Project.
4. System displays the project form.
5. Manager enters project name, description, status, and other required information.
6. Manager submits the form.
7. Backend validates the request.
8. Backend checks authorization and SaaS limits.
9. System creates the Project record.
10. System returns a successful response.
11. Frontend displays the new project.

Alternative / Exception Flows:

- Required field missing → validation error is returned.
- User lacks permission → request is rejected.
- Organization plan limit reached → project is not created and an appropriate limit message is returned.
- Database/service failure → operation fails safely and an appropriate error is logged/returned.

Postcondition: A valid project exists under the correct organization and is visible only to authorized users.

4.3 Core Codentraa Use Cases

ID	Use Case	Actor	Purpose
UC-AUTH-001	Register Account	User	Create a Codentraa account.
UC-AUTH-002	Login	User	Authenticate and access the application.
UC-ORG-001	Create Organization	Owner	Create a workspace/tenant.
UC-MEM-001	Invite Member	Owner/Admin	Invite a user to an organization.
UC-MEM-002	Manage Role	Owner/Admin	Change a member's role.
UC-PROJ-001	Create Project	Manager	Create a project.
UC-PROJ-002	Manage Project	Manager	Edit/archive/manage a project.
UC-TASK-001	Create Task	Manager/Member	Create a project task.
UC-TASK-002	Assign Task	Manager	Assign a task to an eligible member.

UC-TASK-003	Update Task	Member	Update task status/details.
UC-COL-001	Comment on Task	Member	Add a task comment.
UC-COL-002	Upload Attachment	Member	Attach a permitted file.
UC-NOT-001	Receive Notification	User	Receive and read relevant notifications.
UC-DASH-001	View Dashboard	User	View authorized workspace/project information.
UC-REP-001	View Reports	Manager	View project/team analytics.
UC-SUB-001	Manage Subscription	Owner	View/change subscription.

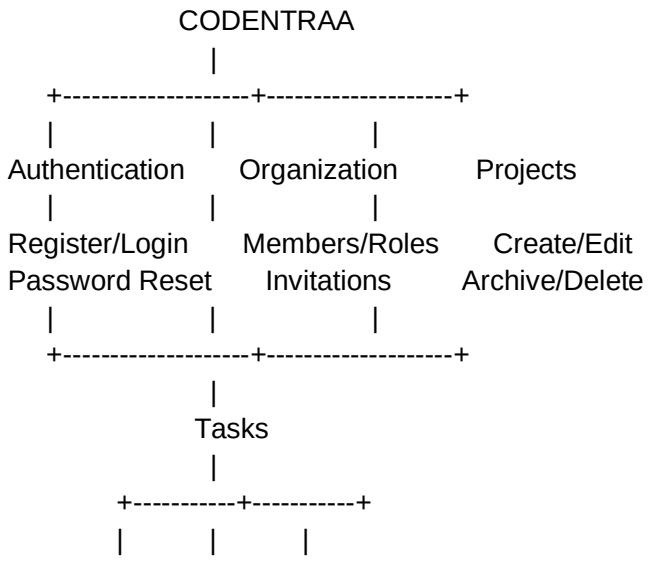
5. Step 4 — Use Case Diagram

The Use Case Diagram provides a visual, high-level view of which actors interact with which major Codentraa functions. It summarizes the detailed Use Cases.

5.1 Actors

- Owner — organization ownership, members, projects, subscription, settings.
- Admin — organization/member administration and permitted settings.
- Manager — projects, tasks, team workload, reports.
- Developer/Member — assigned work, task updates, comments, attachments.
- Client — permitted project visibility and collaboration.
- Payment Provider — external billing interaction.
- Email Service — invitations, verification, password reset, and notifications.

5.2 Text-Based Use Case Diagram



Create Assign Update
|
Comments / Attachments
|
Dashboard / Reports
|
Subscription / Billing

In a graphical UML diagram, actors are placed outside the Codentraa system boundary and use cases inside the boundary. Associations connect actors to the actions they are allowed to perform.

6. Step 5 — Database Design / ERD

The ERD translates system requirements and use cases into persistent data. The most important database rule for Codentraa is tenant isolation: organization-owned records must be traceable to the correct organization.

6.1 Core Entities

Entity	Purpose
User	Stores user identity and account information.
Organization	Represents a Codentraa workspace/tenant.
OrganizationMember	Links users to organizations and their roles.
Role	Defines permissions/role information.
Project	Stores organization projects.
ProjectMember	Links eligible members to projects.
Task	Stores work items belonging to projects.
Comment	Stores task/project discussion.
Attachment	Stores metadata for uploaded files.
Notification	Stores user notifications.
Plan	Stores SaaS plan definitions and limits.
Subscription	Stores an organization's subscription state.
Payment	Stores billing/payment records as appropriate.
AuditLog	Stores important security/business activity.

6.2 Main Relationships

- User 1 → many OrganizationMember records.
- Organization 1 → many OrganizationMember records.

- Organization 1 → many Projects.
- Project 1 → many ProjectMember records.
- Project 1 → many Tasks.
- Task 1 → many Comments.
- Task 1 → many Attachments.
- User 1 → many Notifications.
- Organization 1 → one or more Subscription records over its lifecycle.
- Subscription → Payment records according to the billing model.
- Organization/User/Resource → AuditLog records as required.

6.3 Text-Based ERD

```
USER
 |
 +----< ORGANIZATION_MEMBER >---- ORGANIZATION
                                  |
                                  +----< PROJECT
                                         |
                                         +----< PROJECT_MEMBER
                                         |
                                         +----< TASK
                                               |
                                               +----< COMMENT
                                               |
                                               +----< ATTACHMENT

ORGANIZATION ----< SUBSCRIPTION ----< PAYMENT
USER ----------< NOTIFICATION
USER/ORG/RESOURCE ----< AUDIT_LOG
```

6.4 Tenant Isolation

The backend must ensure that a user can access a resource only when the user has access to the resource's organization. Do not authorize a request using a project ID alone.

Secure request pattern: Authenticate User → Determine Organization Context → Verify Resource Organization → Verify Permission → Execute Operation.

7. Step 6 — System Architecture

System architecture defines how Codentraa's frontend, backend, database, storage, external services, and infrastructure communicate.

7.1 Recommended High-Level Architecture

```
USER
 ↓
WEB BROWSER
 ↓
NEXT.JS FRONTEND
 ↓ HTTPS / JSON
```

ASP.NET CORE WEB API
↓
AUTHENTICATION + AUTHORIZATION
↓
APPLICATION / BUSINESS LOGIC
↓
ENTITY FRAMEWORK CORE
↓
SQL SERVER

Additional services:
API → Object/File Storage
API → Email Service
API → Payment Provider
API → Logging/Monitoring

7.2 Architecture Layers

Layer	Responsibility
Presentation Layer	Next.js UI, forms, pages, client-side state, loading/error states.
API Layer	HTTP endpoints, request validation, authentication/authorization boundaries.
Application/Business Layer	Use-case orchestration and business rules such as project limits and permissions.
Domain/Data Layer	Entities, data access, EF Core, and database operations.
Infrastructure Layer	Email, file storage, payment provider, logging, background jobs, external integrations.
Database	SQL Server storing relational application data.

7.3 Example End-to-End Request — Create Project

12. Manager submits the Create Project form in Next.js.
13. Frontend sends POST /api/projects with project data.
14. API authenticates the request.
15. API determines the active organization.
16. Authorization checks whether the Manager can create projects.
17. Business logic checks subscription/project limits.
18. Application validates project data.
19. EF Core creates the Project record in SQL Server with the correct OrganizationId.
20. API returns a success response.
21. Frontend updates the project list and shows success feedback.
22. Audit logging records the event if required.

8. How Steps 3–6 Connect

USER STORY
↓
USE CASE
↓
USE CASE DIAGRAM
↓
DATA REQUIREMENTS
↓
ERD / DATABASE
↓
SYSTEM ARCHITECTURE
↓
API + UI IMPLEMENTATION

Example: “As a Manager, I want to create a project.” The Use Case describes the interaction. The Use Case Diagram shows the Manager connected to Create Project. The ERD identifies Organization and Project data. The architecture defines how the Next.js page calls the ASP.NET Core API, how business rules are applied, and how the Project is stored in SQL Server.

9. Design Rules Before Coding

- Do not design database tables without first understanding the required use cases.
- Do not expose an API endpoint without defining its authorization rules.
- Do not rely on frontend permissions for security.
- Every tenant-owned resource must have a clear organization boundary.
- Every important use case should have testable acceptance criteria.
- Keep business rules in the backend/application layer.
- Document external services and failure behavior.
- Keep MVP scope controlled; advanced AI/integrations can follow after the core product is stable.

10. Deliverables From This Phase

- Detailed Use Case Specification
- Use Case Diagram
- Entity List
- ERD
- Database Schema Draft
- System Architecture Diagram
- Module/Layer Boundaries
- End-to-End Feature Flow

11. Next Step After Steps 3–6

Once these four design artifacts are approved, the next step is Step 7 — API Design. Each important use case will be mapped to concrete endpoints, request/response models, authentication requirements,

authorization rules, validation, status codes, and error handling. After API design, move to Figma/UI design and then implementation.

CODENTRAA
Step 7 — API Design & Specification

Version 1.0
SaaS Project & Team Management Platform

Prepared By: Muhammad Atif
Date: August 2026
Status: Draft

1. Purpose

This document defines the API contract for Codentraa. It translates the approved SRS, User Stories, Use Cases, database design, and system architecture into concrete backend endpoints that the frontend and other authorized clients can consume.

2. API Position in the Development Cycle

SRS → User Stories → Use Cases → Use Case Diagram → ERD → System Architecture → API Design → Figma/UI → Development

The API is the contract between the presentation layer and backend application. It should expose business capabilities, not database tables directly.

3. Recommended Technology

- Backend: ASP.NET Core Web API.
- Language: C#.
- ORM/Data Access: Entity Framework Core.
- Database: SQL Server.
- API format: REST over HTTPS with JSON.
- API documentation/testing: OpenAPI/Swagger.
- Authentication: secure token/session-based authentication appropriate to the final deployment model.
- Authorization: server-side role/permission checks.
- Versioning: /api/v1 as the initial public API namespace.

4. Base API Structure

Base URL pattern:

https://api.codentraa.example/api/v1

Example endpoint:

POST /api/v1/projects

5. Standard Request / Response Rules

Request:

```
{
  "name": "Codentraa Website",
  "description": "Main website project"
}
```

Successful response:

```
{
  "success": true,
  "message": "Project created successfully.",
  "data": {
```

```
    "id": "project-id",
    "name": "Codentraa Website"
  }
}
```

Error response:

```
{
  "success": false,
  "message": "Validation failed.",
  "errors": {
    "name": ["Project name is required."]
  }
}
```

6. HTTP Methods

Method	Typical Use	Example
GET	Read data	GET /projects
POST	Create a resource/action	POST /projects
PUT	Replace/update a resource	PUT /projects/{id}
PATCH	Partially update a resource	PATCH /projects/{id}
DELETE	Delete/remove a resource	DELETE /projects/{id}

7. HTTP Status Codes

Code	Meaning
200	Successful request
201	Resource created
204	Successful request with no response body
400	Bad request/validation problem
401	Not authenticated
403	Authenticated but not authorized
404	Resource not found or not accessible
409	Conflict, such as duplicate data
422	Semantically invalid request when used by the API policy
429	Rate limit exceeded

500	Unexpected server error

8. Authentication API

Method	Endpoint	Access	Purpose
POST	/auth/register	Public	Create user account.
POST	/auth/login	Public	Authenticate user.
POST	/auth/logout	Authenticated	Terminate current session/token according to auth model.
POST	/auth/verify-email	Public/Token	Verify user email.
POST	/auth/forgot-password	Public	Request password recovery.
POST	/auth/reset-password	Public/Token	Set a new password.

9. Organization API

Method	Endpoint	Access	Purpose
POST	/organizations	Authenticated	Create organization.
GET	/organizations	Authenticated	List organizations available to the current user.
GET	/organizations/{organizationId}	Member	Get authorized organization details.
PATCH	/organizations/{organizationId}	Owner/Admin	Update allowed organization settings.
DELETE	/organizations/{organizationId}	Owner	Delete/deactivate according to retention policy.

10. Member & Role API

Method	Endpoint	Access	Purpose
GET	/organizations/{organizationId}/members	Member	List authorized members.
POST	/organizations/{organizationId}/invitations	Owner/Admin	Invite member.
POST	/invitations/{token}/accept	Invited User	Accept invitation.

PATCH	/organizations/{organizationId}/members/{userId}/role	Owner/Admin	Change member role.
DELETE	/organizations/{organizationId}/members/{userId}	Owner/Admin	Remove member.

11. Project API

Method	Endpoint	Access	Purpose
GET	/organizations/{organizationId}/projects	Member	List accessible projects.
POST	/organizations/{organizationId}/projects	Manager	Create project.
GET	/projects/{projectId}	Project Member	Get project details.
PATCH	/projects/{projectId}	Manager	Update project.
POST	/projects/{projectId}/members	Manager	Add project member.
DELETE	/projects/{projectId}/members/{userId}	Manager	Remove project member.
POST	/projects/{projectId}/archive	Manager	Archive project.
DELETE	/projects/{projectId}	Owner/Admin	Delete project when permitted.

12. Task API

Method	Endpoint	Access	Purpose
GET	/projects/{projectId}/tasks	Project Member	List tasks.
POST	/projects/{projectId}/tasks	Manager/Member	Create task.
GET	/tasks/{taskId}	Authorized User	Get task.
PATCH	/tasks/{taskId}	Authorized User	Update task.
PATCH	/tasks/{taskId}/status	Authorized User	Change status.
PATCH	/tasks/{taskId}/assignee	Manager	Assign/reassign task.
DELETE	/tasks/{taskId}	Manager/Owner	Delete task.

13. Collaboration API

Method	Endpoint	Access	Purpose

GET	/tasks/{taskId}/comments	Authorized User	List comments.
POST	/tasks/{taskId}/comments	Authorized User	Create comment.
PATCH	/comments/{commentId}	Comment Author/Authorized Role	Edit comment when permitted.
DELETE	/comments/{commentId}	Comment Author/Admin	Delete comment when permitted.
POST	/tasks/{taskId}/attachments	Authorized User	Upload attachment.
GET	/attachments/{attachmentId}	Authorized User	Access attachment metadata/download flow.

14. Notification API

Method	Endpoint	Access	Purpose
GET	/notifications	Authenticated	List current user's notifications.
GET	/notifications/unread-count	Authenticated	Get unread count.
PATCH	/notifications/{notificationId}/read	Authenticated	Mark notification as read.
PATCH	/notifications/read-all	Authenticated	Mark all eligible notifications as read.

15. Dashboard & Reports API

Method	Endpoint	Access	Purpose
GET	/organizations/{organizationId}/dashboard	Member/Manager	Return dashboard metrics allowed for the user.
GET	/projects/{projectId}/progress	Project Member/Manager	Return project progress data.
GET	/organizations/{organizationId}/reports/workload	Manager	Return team workload.
GET	/organizations/{organizationId}/reports/overdue-tasks	Manager	Return overdue tasks.

16. SaaS Subscription & Billing API

Method	Endpoint	Access	Purpose

GET	/plans	Public/Authenticated	List available plans.
GET	/organizations/{organizationId}/subscription	Owner/Admin	Get current subscription.
POST	/organizations/{organizationId}/subscription/checkout	Owner	Create billing checkout session.
POST	/organizations/{organizationId}/subscription/upgrade	Owner	Request plan upgrade.
POST	/organizations/{organizationId}/subscription/cancel	Owner	Request cancellation.
POST	/billing/webhooks/provider	External Provider	Receive verified billing events.

17. Create Project API — Complete Example

Endpoint: POST /api/v1/organizations/{organizationId}/projects

Request body:

```
{
  "name": "Codentraa Website",
  "description": "Main website project",
  "status": "Active"
}
```

Processing flow:

1. Authenticate request.
2. Validate organizationId and request body.
3. Verify the user belongs to the organization.
4. Verify the user's project-creation permission.
5. Check subscription/project limits.
6. Create the Project entity with the correct OrganizationId and creator information.
7. Save through Entity Framework Core.
8. Create an audit record if required.
9. Return the created project.

Success: HTTP 201 Created.

Possible errors: 400 validation error, 401 unauthenticated, 403 unauthorized, 404 organization unavailable, 409 business conflict, 429 limit/rate issue, 500 unexpected error.

18. Authorization & Role Matrix

Feature	Owner	Admin	Manager	Member	Client

Organization Settings	Full	Allowed	View	View	No
Members	Full	Manage	View	View	Restricted
Projects	Full	Manage	Manage	Assigned/Allowed	Allowed
Tasks	Full	Manage	Manage	Assigned/Allowed	Allowed
Reports	Full	Allowed	Manage	Limited	Limited
Billing	Full	No/Restricted	No	No	No

19. Validation Rules

- Required fields must be validated server-side.
- String length limits must be enforced.
- IDs must be validated before database queries.
- Enum/status values must be restricted to supported values.
- Users must not be able to assign tasks to unauthorized users.
- Project operations must verify organization membership.
- Subscription limits must be checked on the backend.
- Uploaded files must be validated for size/type and handled securely.
- Do not return sensitive internal exception details to clients.

20. Security Requirements

- All production API traffic must use HTTPS.
- Authentication credentials/tokens must be handled securely.
- Authorization must be enforced on every protected endpoint.
- Never trust organizationId, userId, role, or permission values supplied by the client.
- Prevent cross-tenant data access.
- Use parameterized/ORM-based data access to reduce injection risk.
- Apply rate limiting where appropriate, especially authentication and public endpoints.
- Validate uploads and restrict access to private files.
- Do not expose passwords, secrets, payment credentials, or internal stack traces in API responses.
- Log security-relevant events without storing unnecessary sensitive data.

21. Pagination, Filtering & Sorting

For collection endpoints, use a consistent query pattern, for example:

GET /api/v1/organizations/{organizationId}/projects?page=1&pageSize=20&status=Active&sortBy=createdAt&sortOrder=desc

Recommended response metadata:

```
{
  "success": true,
  "data": [],
```

```
  "pagination": {
    "page": 1,
    "pageSize": 20,
    "totalItems": 100,
    "totalPages": 5
  }
}
```

22. API ↔ Frontend ↔ Database Flow

```
NEXT.JS UI
  ↓ HTTPS / JSON
API CONTROLLER
  ↓
AUTHENTICATION
  ↓
AUTHORIZATION
  ↓
APPLICATION / BUSINESS LOGIC
  ↓
EF CORE / DATA ACCESS
  ↓
SQL SERVER
  ↓
RESPONSE
  ↑
NEXT.JS UI
```

The frontend should not connect directly to SQL Server. The API is responsible for authorization, business rules, validation, data access, and controlled responses.

23. API Development Checklist

- Endpoint has a clear purpose.
- HTTP method is appropriate.
- Authentication requirement is defined.
- Authorization requirement is defined.
- Request DTO is defined.
- Validation rules are defined.
- Success response is defined.
- Error responses are defined.
- Database interaction is defined.
- Tenant isolation is verified.
- Logging/auditing requirement is defined.
- API is documented in OpenAPI/Swagger.
- Automated tests cover important behavior.

24. Next Step — Step 8 Figma / UI Design

After the API contract is approved, Step 8 is Figma/UI Design. Each API-driven use case should be mapped to screens, components, forms, buttons, loading states, empty states, validation, error states, and responsive layouts. The Figma design then becomes the visual blueprint for the frontend implementation.

CODENTRAA
Step 8 — Figma / UI Design Specification

Version 1.0
SaaS Project & Team Management Platform

Prepared By: Muhammad Atif
Date: August 2026
Status: Design Blueprint

1. Purpose

This document defines the complete Figma/UI design process for Codentraa. It converts the approved SRS, User Stories, Use Cases, ERD, System Architecture, and API Design into a visual product blueprint that can be implemented by the frontend development team.

2. Position in the Development Cycle

SRS → User Stories → Use Cases → Use Case Diagram → ERD → System Architecture → API Design → FIGMA/UI DESIGN → Development → Testing → Deployment

3. What You Create in Figma

- Information architecture and navigation.
- Wireframes for important screens.
- High-fidelity desktop and mobile screens.
- Reusable UI components.
- Design tokens and visual styles.
- User flows and clickable prototype.
- Forms and validation states.
- Loading, empty, success, and error states.
- Role-based UI variations.
- Responsive layouts.
- Developer handoff specifications.

4. Recommended Figma File Structure

Figma Page	Contents
00 — Cover	Project title, version, owner, design status.
01 — User Flows	Main user journeys and navigation flows.
02 — Wireframes	Low-fidelity layouts before visual styling.
03 — Design System	Colors, typography, spacing, icons, buttons, inputs, cards, tables.
04 — Authentication	Login, register, verification, password recovery.
05 — Dashboard	Main organization dashboard and widgets.
06 — Projects	Project list, create/edit, project details.
07 — Tasks	Task list/board, task details, create/edit.
08 — Team & Members	Members, roles, invitations.
09 — Collaboration	Comments, attachments, notifications.
10 — Reports	Analytics and reporting screens.

11 — Billing	Plans, subscription, billing status.
12 — Settings	Profile, organization, security and preferences.
13 — Prototype	Clickable end-to-end flows.
14 — Handoff	Final screens, annotations and developer notes.

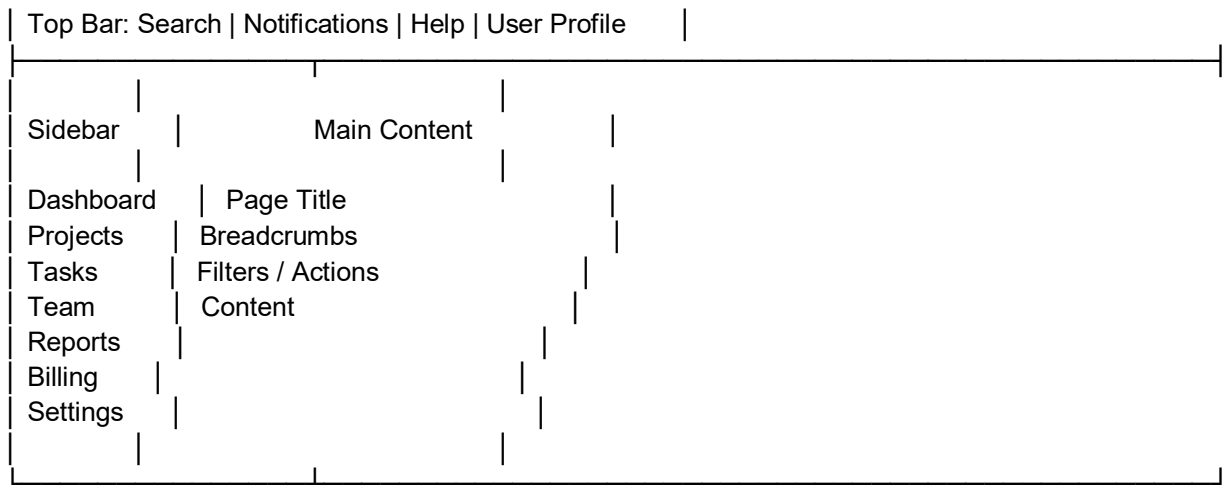
5. Information Architecture

CODENTRAA

```
│
├── Dashboard
│
├── Projects
│   ├── All Projects
│   ├── Create Project
│   └── Project Details
│       ├── Overview
│       ├── Tasks
│       ├── Members
│       └── Activity
│
├── Tasks
│   ├── My Tasks
│   ├── All Tasks
│   └── Task Details
│
├── Team
│   ├── Members
│   ├── Invitations
│   └── Roles
│
├── Reports
│
├── Notifications
│
├── Billing / Subscription
│
└── Settings
    ├── Profile
    ├── Organization
    ├── Security
    └── Preferences
```

6. Global Application Layout

DESKTOP APP SHELL



On mobile, the sidebar should collapse into a menu/drawer and the main content should become a single-column responsive layout.

7. Authentication Screens

ID	Screen	Main Elements
AUTH-01	Login	Email/username, password, remember option, login button, forgot password, register link.
AUTH-02	Register	Name, email, password, confirm password, terms acceptance, create account.
AUTH-03	Email Verification	Verification status, resend verification option.
AUTH-04	Forgot Password	Email field and password-reset request.
AUTH-05	Reset Password	New password, confirm password, submit.
AUTH-06	Invitation Acceptance	Invitation details, sign in/register, accept invitation.

8. Dashboard Screens

- Organization name and active workspace selector.
- Summary cards: active projects, open tasks, overdue tasks, team members.
- Project progress overview.
- My assigned tasks.
- Recent activity.
- Notifications preview.

- Quick actions: Create Project, Create Task, Invite Member.
- Optional subscription/usage indicator.

9. Projects UI

ID	Screen	Main Elements
PROJ-01	Project List	Search, filters, status, project cards/table, create project button.
PROJ-02	Create Project	Project name, description, status, optional dates, submit/cancel.
PROJ-03	Project Details	Overview, progress, tasks, members, activity.
PROJ-04	Edit Project	Editable project information.
PROJ-05	Archive Confirmation	Confirmation modal and warning.

10. Tasks UI

- Task list with search, filters, sorting and pagination.
- Optional Kanban board: Backlog → In Progress → Review → Done.
- Create Task modal/page.
- Task detail panel/page.
- Task title, description, status, priority, assignee, due date and project.
- Comments and activity timeline.
- Attachments area.
- Edit/delete actions according to role.
- Responsive mobile task view.

11. Team & Members UI

- Members table with name, email, role, status and actions.
- Invite Member modal.
- Pending invitations list.
- Role selection/update interface.
- Remove member confirmation.
- Permission-aware action visibility.
- Empty state when no members exist.

12. Reports UI

- Project progress summary.
- Task status distribution.
- Completed vs pending tasks.
- Overdue tasks.
- Team workload.

- Date/project filters.
- Charts and summary cards.
- Export action if included in the approved product scope.

13. Billing / Subscription UI

- Current plan card.
- Available plan comparison.
- Usage/limit indicators.
- Upgrade button.
- Checkout state.
- Subscription status.
- Cancellation flow.
- Payment/billing history when supported.
- Billing permissions visible only to authorized roles.

14. Settings UI

- Profile information.
- Password/security settings.
- Organization settings.
- Notification preferences.
- Workspace preferences.
- Subscription/billing shortcut for owners.
- Account logout.

15. Design System

Before designing all screens, create reusable components and styles. The exact brand colors can be finalized in Figma; consistency is more important than choosing many colors.

- Typography: define heading, body, caption and label styles.
- Spacing: establish a consistent spacing scale.
- Grid: define desktop and mobile layout rules.
- Buttons: primary, secondary, text, destructive, disabled and loading states.
- Inputs: default, focus, error, success, disabled.
- Cards: dashboard, project, task and plan cards.
- Tables: header, row, selected, empty and pagination states.
- Modals: confirmation, form and information modal.
- Badges: status, role, priority and subscription state.
- Navigation: sidebar, top bar, breadcrumbs and mobile navigation.
- Icons: use one consistent icon family.
- Toast/alerts: success, warning, error and informational feedback.

16. UI States — Mandatory

State	Design Requirement

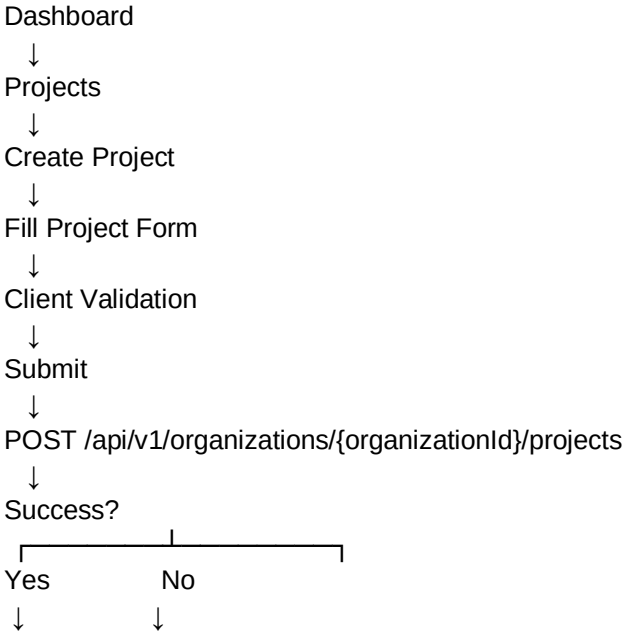
Default	Normal loaded state.
Loading	Skeleton/spinner while data is being fetched.
Empty	No records exist; explain why and provide a useful action.
Error	Explain failure and provide retry/recovery.
Success	Confirm completed action.
Validation Error	Show field-level and/or form-level validation.
Disabled	Show unavailable action without misleading the user.
Permission Restricted	Hide or disable actions the user cannot perform.
Offline/Network Failure	Provide an appropriate recovery message where relevant.

17. Responsive Design

Design at least desktop and mobile layouts. Tablet behavior should be derived from the responsive system rather than creating unnecessary separate screens.

- Desktop: persistent sidebar + content area.
- Tablet: collapsible sidebar and adaptable tables/cards.
- Mobile: drawer navigation, single-column content, stacked forms and horizontally scrollable data where necessary.
- Buttons and touch targets must be usable on mobile.
- Important actions should remain easy to reach on small screens.

18. User Flow — Create Project



Project Details Error State
↓
Project List Updated

19. User Flow — Create & Complete Task

Project Details
↓
Tasks
↓
Create Task
↓
Enter Task Information
↓
Assign Member
↓
Save
↓
Task Appears in List/Board
↓
Member Opens Task
↓
Updates Status
↓
Done

20. API Mapping to UI

UI Screen	API	Purpose
Login Screen	POST /api/v1/auth/login	Authenticate user.
Register Screen	POST /api/v1/auth/register	Create account.
Project List	GET /api/v1/organizations/{organizationId}/projects	Load projects.
Create Project	POST /api/v1/organizations/{organizationId}/projects	Create project.
Project Details	GET /api/v1/projects/{projectId}	Load project.
Task List	GET /api/v1/projects/{projectId}/tasks	Load tasks.
Create Task	POST /api/v1/projects/{projectId}/tasks	Create task.
Members	GET /api/v1/organizations/{organizationId}/members	Load members.
Invite Member	POST	Invite member.

	/api/v1/organizations/{organizationId}/invitations	
Notifications	GET /api/v1/notifications	Load notifications.
Dashboard	GET /api/v1/organizations/{organizationId}/dashboard	Load metrics.
Subscription	GET /api/v1/organizations/{organizationId}/subscription	Load plan status.

21. Role-Based UI Behavior

The frontend should reflect permissions for usability, but backend authorization remains the actual security boundary.

UI Area	Owner	Admin	Manager	Member
Billing	Full	Restricted/No	No	No
Members	Full	Manage	View	View
Projects	Full	Manage	Manage	Allowed
Reports	Full	Allowed	Manage	Limited
Organization Settings	Full	Manage	View	No

22. Prototype Requirements

- Create a clickable Login → Dashboard flow.
- Create Dashboard → Projects → Create Project → Project Details flow.
- Create Project → Task → Assign Member → Update Status flow.
- Create Team → Invite Member → Accept Invitation flow.
- Create Notifications → Read Notification flow.
- Create Subscription → Plan → Checkout flow if billing is included in the MVP.
- Connect primary navigation between major modules.

23. Figma Naming Convention

- Pages: 01_User_Flows, 02_Wireframes, 03_Design_System, etc.
- Frames: AUTH-01_Login, DASH-01_Overview, PROJ-01_Project_List.
- Components: Button/Primary, Input/Text, Modal/Confirmation, Card/Project.
- Variants: Button = Default/Hover/Disabled/Loading.
- Keep component names consistent so developers can understand them quickly.

24. Developer Handoff

- Every final screen has a clear frame name.
- Reusable components are componentized.

- Spacing, typography and component behavior are documented.
- Desktop and mobile layouts are provided.
- Interactive states are included.
- Assets/icons are organized.
- API actions are mapped to UI actions.
- Acceptance criteria are traceable to the final design.
- Prototype links demonstrate important workflows.

25. Figma Completion Checklist

□ Information architecture completed

□ User flows completed

□ Wireframes completed

□ Design system created

□ Authentication screens completed

□ Dashboard completed

□ Project screens completed

□ Task screens completed

□ Team/member screens completed

□ Reports completed

□ Billing completed if included in MVP

□ Settings completed

□ Loading/empty/error/success states completed

□ Responsive layouts completed

□ Prototype connected

□ API mapping reviewed

□ Developer handoff completed

26. What Happens After Figma

Once Step 8 is approved, the project enters Step 9 — Development. The developer uses the SRS for requirements, Use Cases for behavior, ERD for data, Architecture for technical boundaries, API Design for backend contracts, and Figma for the visual implementation.

FINAL IMPLEMENTATION INPUTS

SRS
+ User Stories
+ Use Cases

+ Use Case Diagram
+ ERD
+ Architecture
+ API Specification
+ Figma

↓

DEVELOPMENT

CODENTRAA

Step 9 — Development Process & Implementation Plan

Version 1.0

SaaS Project & Team Management Platform

Prepared By: Muhammad Atif

Date: August 2026

Status: Development Blueprint

1. Purpose

This document defines how Codentraa moves from approved requirements and design into a working software product. It provides a practical development sequence, project structure, feature-by-feature implementation strategy, development rules, testing checkpoints, and the transition toward production.

2. Development Cycle Position

SRS → User Stories → Use Cases → Use Case Diagram → ERD → System Architecture → API Design → Figma/UI Design → DEVELOPMENT → Testing → Deployment → Maintenance

3. What Is Created During Development?

- Frontend application using the approved Figma design.
- Backend Web API implementing the approved API contract.
- SQL Server database implementing the approved ERD.
- Authentication and authorization system.
- Business logic and SaaS rules.
- Project, task, team and collaboration modules.
- API-to-frontend integration.
- Automated and manual tests.
- Production-ready configuration and deployment assets.

4. Recommended Technology Stack

Area	Technology	Purpose
Frontend	Next.js / React	User interface and client application.
Styling	Tailwind CSS or approved UI system	Responsive interface implementation.
Backend	ASP.NET Core Web API	Business logic and REST APIs.
Language	C#	Backend development.
ORM	Entity Framework Core	Database access and migrations.
Database	SQL Server	Relational application data.
API Docs	OpenAPI / Swagger	API documentation and testing.
Source Control	Git / GitHub	Version control and collaboration.
Deployment	Chosen cloud/platform	Production hosting.

5. Development Architecture

CODENTRAA
|

```
├── frontend/
│   └── Next.js / React
│
├── backend/
│   └── ASP.NET Core Web API
│
├── database/
│   └── SQL Server + EF Core
│
├── docs/
│   ├── SRS
│   ├── Use Cases
│   ├── ERD
│   ├── Architecture
│   ├── API Specification
│   └── Figma Specification
│
└── README.md
```

6. Development Strategy

Do not build the complete frontend first and the complete backend afterward. Build Codentraa feature-by-feature. Each feature should move through database → backend/API → frontend → integration → testing before being considered complete.

FEATURE DEVELOPMENT LOOP

Requirement
↓
Database
↓
Backend / API
↓
Frontend
↓
Integration
↓
Testing
↓
Code Review
↓
Feature Complete

7. Phase 1 — Project Setup

1. Create the Git repository.
2. Create the Next.js frontend project.
3. Create the ASP.NET Core Web API project.
4. Configure SQL Server connection.

5. Configure Entity Framework Core.
6. Create development environment configuration.
7. Configure CORS between frontend and backend.
8. Enable Swagger/OpenAPI.
9. Create initial README and development instructions.
10. Create a consistent branching and commit strategy.

8. Phase 2 — Database Development

Convert the approved ERD into actual database entities and relationships.

- User
- Organization
- OrganizationMember
- Role
- Project
- ProjectMember
- Task
- Comment
- Attachment
- Notification
- Plan
- Subscription
- Payment
- AuditLog

Implementation sequence:

11. Create entity classes.
12. Configure primary keys.
13. Configure foreign keys and relationships.
14. Configure required fields and constraints.
15. Create DbContext.
16. Create EF Core migration.
17. Update SQL Server database.
18. Verify relationships and indexes.
19. Add seed data only where appropriate.

9. Phase 3 — Backend Foundation

Backend structure:

```
backend/
├── Controllers/
├── Services/
├── DTOs/
├── Models/
├── Data/
├── Middleware/
├── Authorization/
├── Authentication/
```

└── Configuration/

- Create API versioning/namespacing strategy.
- Create DTOs instead of exposing database entities directly.
- Implement validation.
- Implement global error handling.
- Implement logging.
- Implement authentication middleware.
- Implement authorization policies.
- Configure dependency injection.
- Document endpoints in Swagger.

10. Phase 4 — Authentication & Authorization

AUTHENTICATION FLOW

Register
↓
Email Verification
↓
Login
↓
Authenticated Session / Token
↓
Authorized API Requests

- Register account.
- Verify email where required.
- Login.
- Logout/session termination according to the chosen auth architecture.
- Forgot password.
- Reset password.
- Protect private API endpoints.
- Implement role/permission checks.

Core roles:

- Owner
- Admin
- Manager
- Member
- Client

11. Phase 5 — Core Feature Development

Module	Main Development Scope
Organization	Create organization, view/update settings, workspace

	context.
Members	Invite, accept invitation, list members, change roles, remove members.
Projects	Create, list, view, edit, archive, manage project members.
Tasks	Create, list, view, assign, update, change status, delete.
Collaboration	Comments, attachments, activity and notifications.
Dashboard	Metrics, recent activity, assigned tasks and project summaries.
Reports	Progress, workload, overdue tasks and analytics.

12. Feature-by-Feature Example — Project Module

PROJECT FEATURE

Figma
↓
Project List / Create Form
↓
API Contract
↓
POST /api/v1/organizations/{organizationId}/projects
↓
Controller
↓
Authorization
↓
Project Service
↓
EF Core
↓
SQL Server
↓
201 Created
↓
Frontend updates Project List
↓
Test

20. Create Project page/component from Figma.
21. Implement form validation.
22. Connect Create Project API.
23. Verify organization membership.
24. Verify Manager permission.

25. Check SaaS project limit.
26. Create project record.
27. Return controlled response.
28. Display success/error state.
29. Test authorized and unauthorized cases.

13. Phase 6 — Frontend Development

Suggested frontend structure:

```
frontend/
├── app/
│   ├── login/
│   ├── register/
│   ├── dashboard/
│   ├── projects/
│   ├── tasks/
│   ├── team/
│   ├── reports/
│   ├── billing/
│   └── settings/
├── components/
│   ├── ui/
│   ├── dashboard/
│   ├── projects/
│   ├── tasks/
│   └── team/
├── services/
│   └── api/
└── lib/
```

- Convert approved Figma screens into reusable components.
- Implement routing/navigation.
- Implement forms and client-side validation.
- Connect API service functions.
- Implement loading states.
- Implement empty states.
- Implement error states.
- Implement success feedback.
- Implement responsive layouts.
- Implement role-aware UI behavior.

14. Phase 7 — Frontend ↔ Backend Integration

```
NEXT.JS
  ↓ HTTPS / JSON
ASP.NET CORE API
  ↓
AUTHENTICATION
  ↓
```

AUTHORIZATION
↓
BUSINESS LOGIC
↓
EF CORE
↓
SQL SERVER
↓
JSON RESPONSE
↑
NEXT.JS

The frontend must never connect directly to SQL Server. The API is responsible for authentication, authorization, business rules, validation, and database access.

15. Phase 8 — SaaS Development

Organization / Tenant
↓
Subscription Plan
↓
Usage / Limits
↓
Feature Access
↓
Billing

- Create plan definitions.
- Create subscription model.
- Track organization subscription status.
- Implement plan limits.
- Enforce limits in backend business logic.
- Create plan comparison UI.
- Create subscription management UI.
- Integrate payment provider only after the core product is stable.
- Handle billing webhooks securely.
- Record appropriate billing/audit events.

16. Multi-Tenant Security During Development

Codentraa must treat organization isolation as a core security requirement. Every protected organization-owned resource must be checked against the authenticated user's organization membership and permissions.

SECURE RESOURCE FLOW

Authenticated User
↓
Determine Organization Context
↓

Verify Membership
↓
Verify Permission
↓
Verify Resource Belongs to Organization
↓
Execute Operation

- Never trust organizationId supplied by the browser.
- Never use frontend role checks as the security boundary.
- Never return another tenant's data.
- Apply authorization to every protected endpoint.
- Test cross-tenant access explicitly.

17. Testing During Development

Testing should happen continuously, not only after all coding is finished.

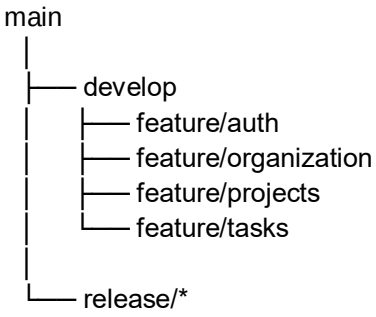
- Unit tests — business rules and services.
- Integration tests — API/database behavior.
- API tests — requests, responses, validation and authorization.
- Frontend tests — important UI behavior.
- End-to-end tests — complete user workflows.
- Security tests — unauthorized and cross-tenant access.
- Responsive tests — desktop, tablet and mobile.

18. Example Test Cases

Feature	Scenario	Expected Result	Status
Register	Valid account	Account created	Pending
Register	Duplicate email	Validation/conflict error	Pending
Login	Correct credentials	Authenticated	Pending
Create Project	Manager with available limit	Project created	Pending
Create Project	Member without permission	Rejected	Pending
Create Project	Plan limit reached	Rejected with limit message	Pending
View Project	Authorized member	Project returned	Pending
View Project	Other tenant user	Rejected / not exposed	Pending

19. Development Workflow / Git

Recommended branch concept:



30. Create a feature branch.
31. Make small logical commits.
32. Run tests and formatting.
33. Review the change.
34. Merge into the development branch.
35. Run integration tests.
36. Promote stable changes toward production.

20. Definition of Done

- Requirement is understood and linked to a User Story/Use Case.
- Database changes are complete.
- API endpoint is implemented and documented.
- Authorization is implemented.
- Frontend UI matches approved Figma.
- Loading/empty/error/success states are handled.
- Frontend and backend integration works.
- Automated/manual tests pass.
- Cross-tenant security has been checked.
- Code is reviewed and documented where necessary.

21. Recommended Development Order

Order	Module	Scope
1	Project setup	Repository, Next.js, ASP.NET Core, SQL Server, EF Core.
2	Database foundation	Entities, relationships, migrations.
3	Authentication	Register, login, password recovery, authorization.
4	Organization	Tenant/workspace creation and context.

5	Members	Invitations and roles.
6	Projects	Project CRUD and members.
7	Tasks	Task CRUD, assignment and status.
8	Collaboration	Comments, attachments, notifications.
9	Dashboard	Real data and metrics.
10	Reports	Progress/workload/overdue reporting.
11	SaaS billing	Plans, subscriptions and payment integration.
12	Hardening	Security, performance, testing and production preparation.

22. First Development Sprint

The first sprint should establish a working technical foundation rather than attempting the full dashboard.

37. Create Git repository.
38. Create Next.js project.
39. Create ASP.NET Core Web API.
40. Configure SQL Server.
41. Configure EF Core.
42. Create initial User, Organization and OrganizationMember entities.
43. Create database migration.
44. Verify API → database connection.
45. Enable Swagger.
46. Create a basic health-check endpoint.
47. Document local setup in README.

23. What Not to Do

- Do not start by building every Figma screen without working APIs.
- Do not connect Next.js directly to SQL Server.
- Do not implement permissions only in the frontend.
- Do not skip database relationships defined in the ERD.
- Do not build billing before the core product is stable.
- Do not wait until the end to test security.
- Do not create large features without breaking them into smaller deliverables.
- Do not change requirements silently; update the relevant documentation when scope changes.

24. Development Completion Flow

REQUIREMENT
↓
DATABASE
↓
BACKEND / API
↓
FRONTEND
↓
INTEGRATION
↓
TESTING
↓
CODE REVIEW
↓
FEATURE COMPLETE
↓
NEXT FEATURE

25. Transition to Testing & Deployment

After the core development backlog is complete, the project moves into formal system testing. Production deployment should only happen after critical functional, security, data, and integration checks have passed.

Development Complete
↓
System Testing
↓
Security Testing
↓
Performance / Reliability Checks
↓
Bug Fixing
↓
Release Candidate
↓
Deployment
↓
Monitoring & Maintenance

26. Final Development Checklist

□ Repository created

□ Frontend created

□ Backend created

□ Database configured

□ EF Core configured

□ Authentication complete

□ Authorization complete

□ Organization complete

□ Members complete

□ Projects complete

□ Tasks complete

□ Collaboration complete

□ Dashboard complete

□ Reports complete

□ Billing complete if included in release

□ Frontend/API integration complete

□ Responsive UI complete

□ Automated/manual tests complete

□ Security checks complete

□ Production configuration prepared

27. Key Principle

Codentraa should be developed as a sequence of working vertical slices. Finish one meaningful feature end-to-end, test it, then move to the next. This reduces integration problems and keeps the project aligned with the SRS, API design, database design, and Figma.